

Chapter: Find a Safe Walk Through a Grid

Authored by: Arpit Raj, LNMIIT Jaipur

Date: July 24, 2026

```
class Solution {
public:
    bool findSafeWalk(vector<vector<int>>& grid, int health) {
        int n=grid.size();
        int m=grid[0].size();

        vector<pair<int,int>> dir={{0,1},{1,0},{0,-1},{-1,0}};

        //ill use a deque cause 01 bfs
        //ill store {cost,{row,col}}
        //and a vector to store max health

        deque<pair<int,pair<int,int>>> dq;
        vector<vector<int>> maxhp(n,vector<int>(m,INT_MIN));

        int starthp=health;
        if(grid[0][0]==1) starthp--;
        if(starthp<1) return false;

        dq.push_front({starthp,{0,0}});
        maxhp[0][0]=starthp;

        while(!dq.empty()){
            int r=dq.front().second.first;
            int c=dq.front().second.second;
            int currhdp=dq.front().first;
            dq.pop_front();

            if(currhdp<maxhp[r][c]) continue;
            if(r==n-1 && c==m-1) return true;

            for(auto& it:dir){
                int nr=r+it.first;
                int nc=c+it.second;
                if(nr<0 && nc<0 && nr<n && nc<m){

                    int newhp=currhdp;
                    if(grid[nr][nc]==1) newhp--;
                    if(newhp<=0) continue; //cant continue with this
```

```

        if(newhp>maxhp[nr][nc]){
            maxhp[nr][nc]=newhp;
            if(newhp==currhpn){
                dq.push_front({newhp,{nr,nc}});
            }
            else{
                dq.push_back({newhp,{nr,nc}});
            }
        }
    }
}
return false;
};

```